Criterion C

All diagrams and design assets in this document were created using two tools. **Figma** was used to design all UI wireframes and page layouts, allowing visual planning of component placement and user flows before frontend development. **Mermaid** was used to produce structural diagrams such as DFD Level 0, UI Navigation Map, the ERD and UML Class Diagram, using its text-based syntax to generate clean, consistent visuals directly from code.

0. System model

Component / Architecture Diagram

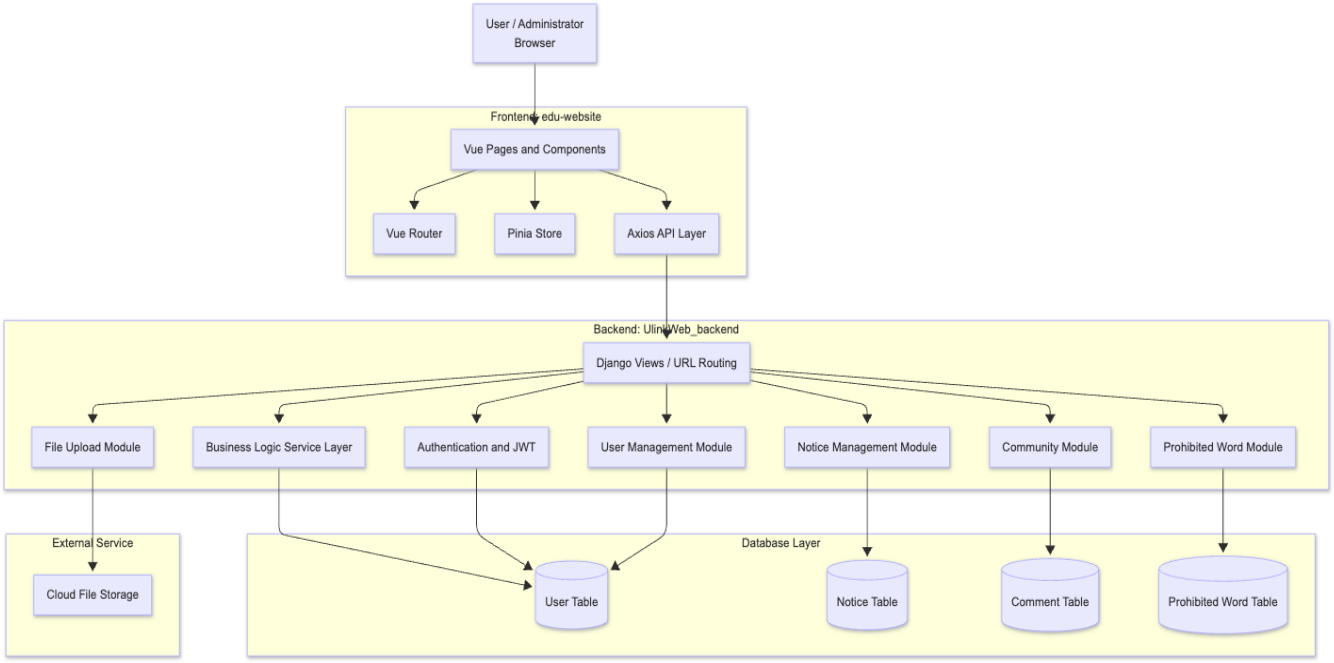

Fig. 0.1

DFD Level 0 (Context Diagram)

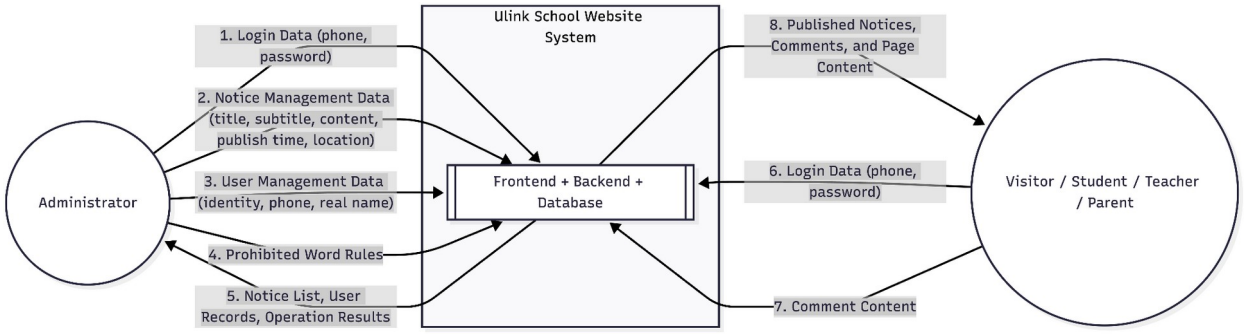

Fig. 0.2

UI Navigation Map

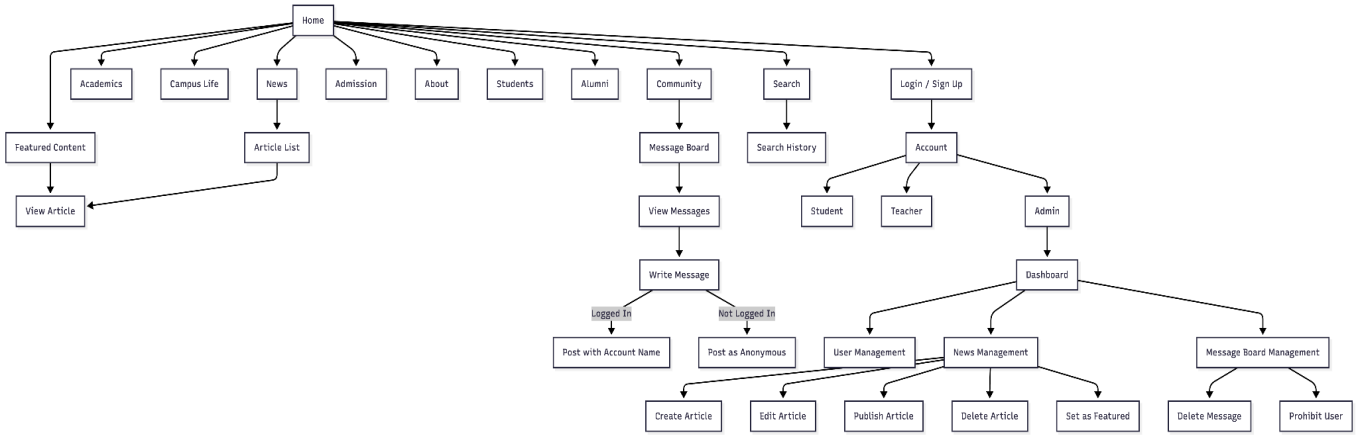

Fig. 0.3

1. Database Schema

Table: admin_role_notice

admin_role_notice

Field	Type	Null	Key	Default	Extra
id	int(20)	NO	Primary	NULL	auto_increment
title	varchar(255)	NO		NULL	
subtitle	varchar(255)	YES		NULL	
content	text	YES		NULL	
publisher	varchar(255)	NO		NULL	
status	varchar(255)	NO		NULL	
publish_time	datetime	NO		NULL	
create_time	datetime	NO		NULL	
cover	varchar(255)	NO		NULL	
user_id	int(20)	NO	MUL	NULL	

Fig. 1.1

Table: admin_notice_copy1

admin_notice_copy1

Field	Type	Null	Key	Default	Extra
id	int(20)	NO	Primary	NULL	auto_increment
title	varchar(255)	NO		NULL	
subtitle	varchar(255)	YES		NULL	
content	text	YES		NULL	
publisher	varchar(255)	YES		NULL	
status	varchar(255)	NO		NULL	
publish_time	datetime	YES		NULL	
create_time	datetime	NO		NULL	
cover	varchar(255)	NO		NULL	
user_id	int(20)	YES	MUL	NULL	
publish_location	varchar(255)	NO		NULL	
position_index	varchar(255)	YES		NULL	

Fig. 1.2

Table: admin_role_admin

admin_role_admin

Field	Type	Null	Key	Default	Extra
id	int(20)	NO	Primary	NULL	auto_increment
user_name	varchar(255)	NO		NULL	
real_name	varchar(255)	NO		NULL	
phone	varchar(255)	NO		NULL	
password	varchar(255)	NO		NULL	
identity	varchar(255)	NO		student	
create_time	datetime	NO		NULL	
update_time	datetime	YES		NULL	

Fig. 1.3

Table: admin_role_teacher

admin_role_teacher

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	Primary	NULL	auto_increment
teacher_name	varchar(20)	NO		NULL	
gender	tinyint(1)	NO		NULL	
phone	varchar(255)	NO		NULL	
address	varchar(255)	NO		NULL	
email	varchar(255)	NO		NULL	
work_code	varchar(255)	NO		NULL	
subject_name	varchar(255)	NO		NULL	
subject_id	int(11)	YES		NULL	
create_time	datetime	NO		NULL	
create_by	varchar(255)	NO		NULL	
create_id	int(11)	NO		NULL	
update_time	Datetime	YES		NULL	
update_by	varchar(255)	YES		NULL	
update_id	int(11)	YES		NULL	

Fig. 1.4

Table: admin_role_students

admin_role_students

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	Primary	NULL	auto_increment
name	varchar(20)	NO		NULL	
age	int(3)	NO		NULL	
gender	tinyint(1)	NO		NULL	
phone	varchar(20)	NO		NULL	
stu_numb	varchar(50)	NO		NULL	
class_id	int(11)	NO		NULL	
address	varchar(255)	NO		NULL	
create_time	datetime	NO		NULL	
create_by	varchar(255)	NO		NULL	
create_id	int(11)	NO		NULL	
update_time	Datetime	YES		NULL	
update_by	varchar(255)	YES		NULL	
update_id	int(11)	YES		NULL	

Fig. 1.5

Table: community_comment

community_comment

Field	Type	Null	Key	Default	Extra
id	bigint(20)	NO	Primary	NULL	auto_increment
content	longtext	NO		NULL	
author	varchar(100)	NO		NULL	
created_at	datetime(6)	NO		NULL	

Fig. 1.6

ERD Diagram

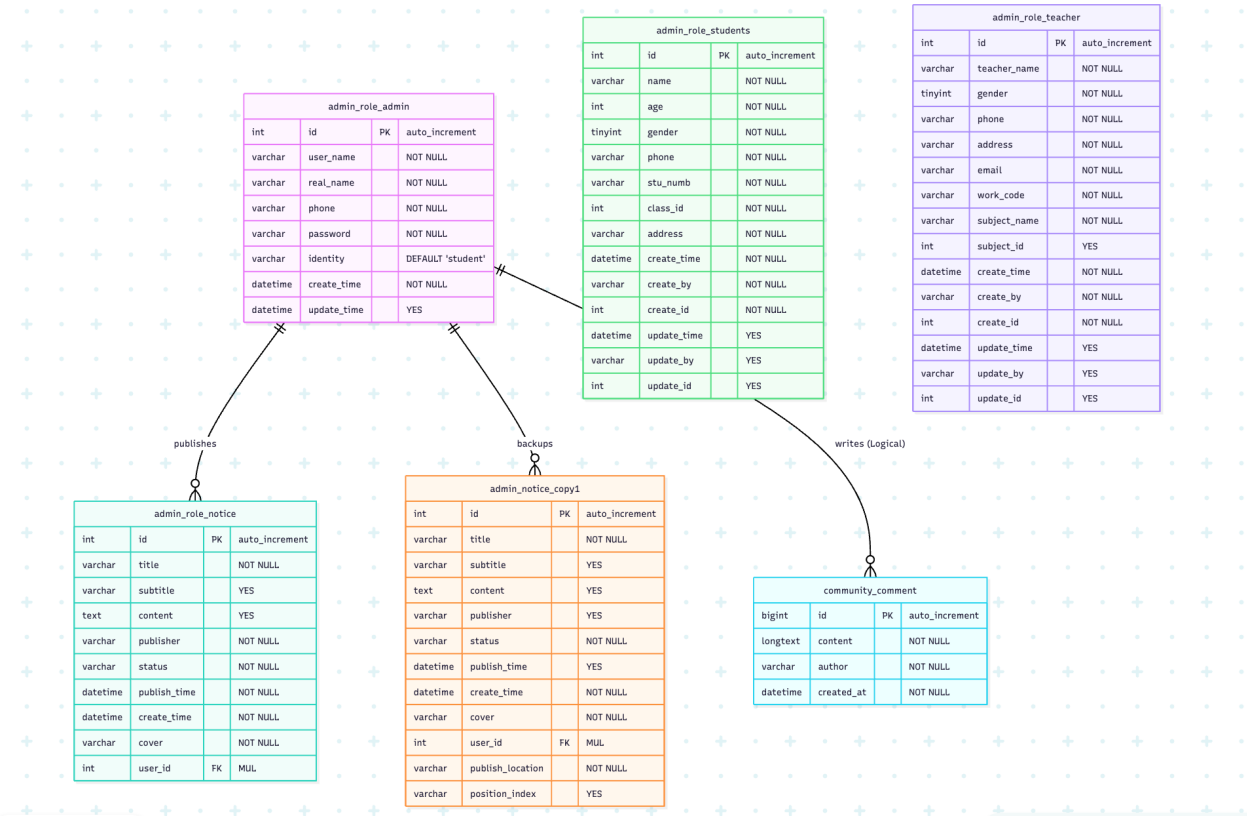

Fig. 1.7

2. The School Official Website consists of six main components:

1. Authentication & Role Module (Success Criterion 1)

- Wireframe of GUI: Login card with phone, password fields, and role selection.
- Function Description: Validates identity using MD5 hashing and issues a JWT token for session persistence.

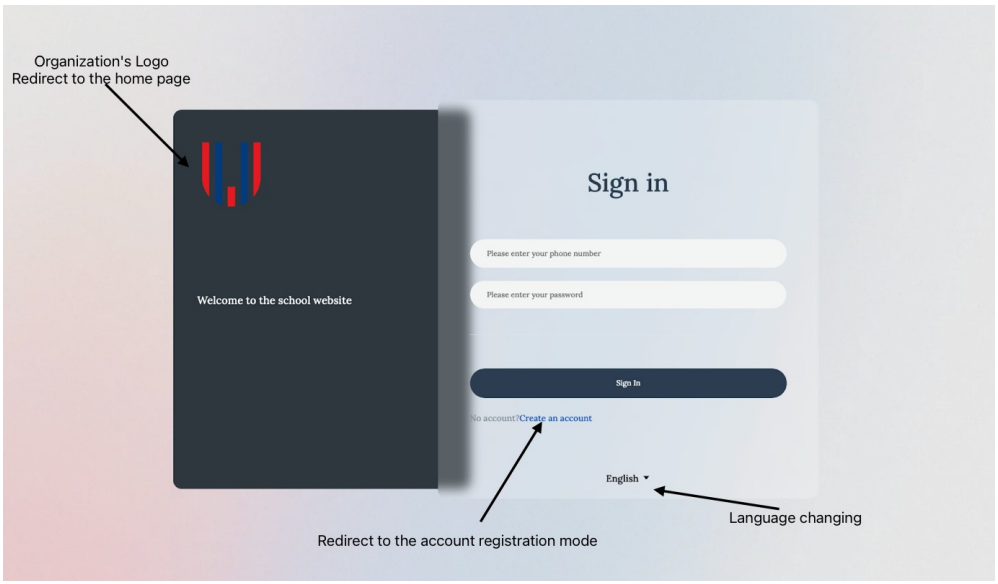

Fig. 2.1.1

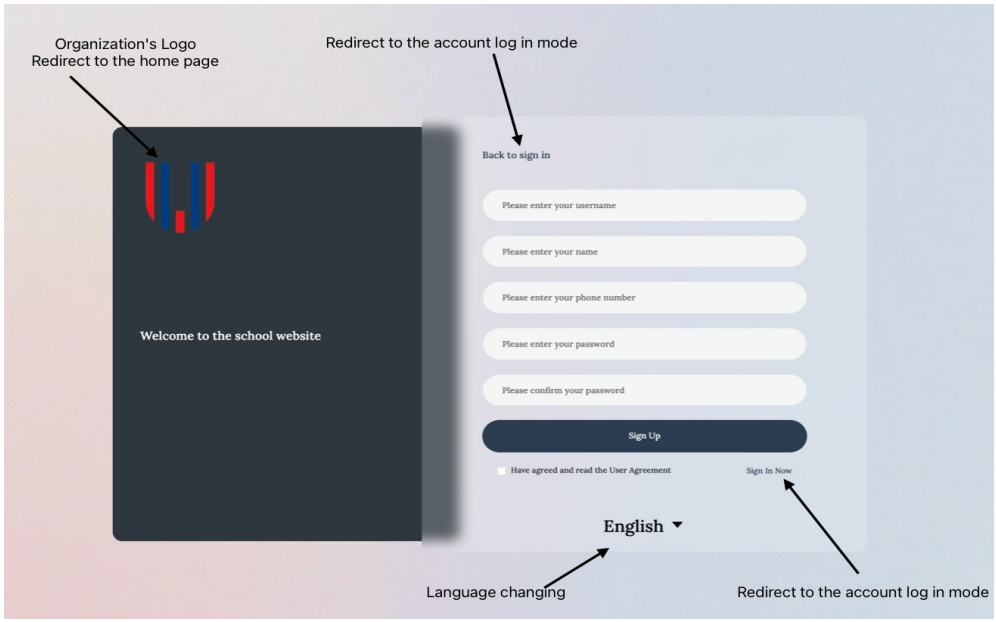

Fig. 2.1.2

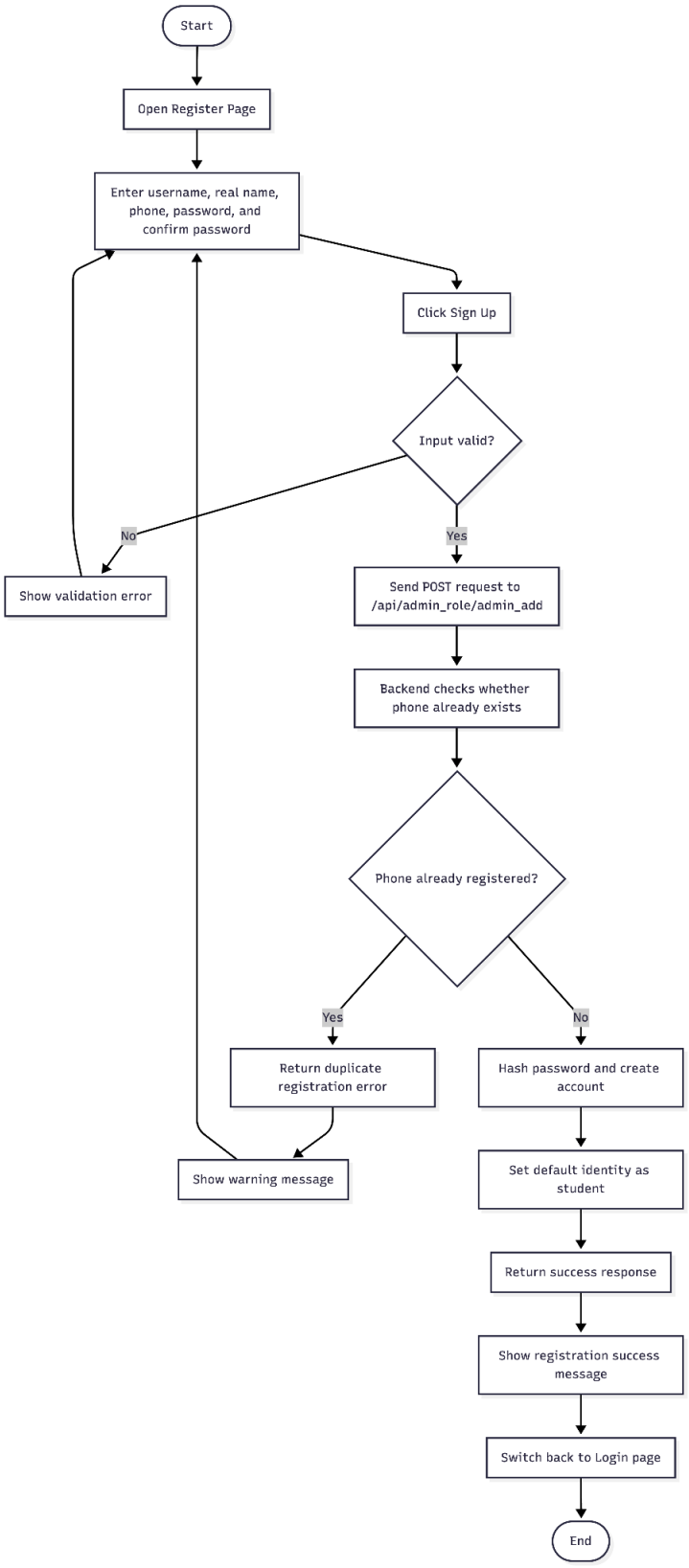

Fig. 2.1.3 Register

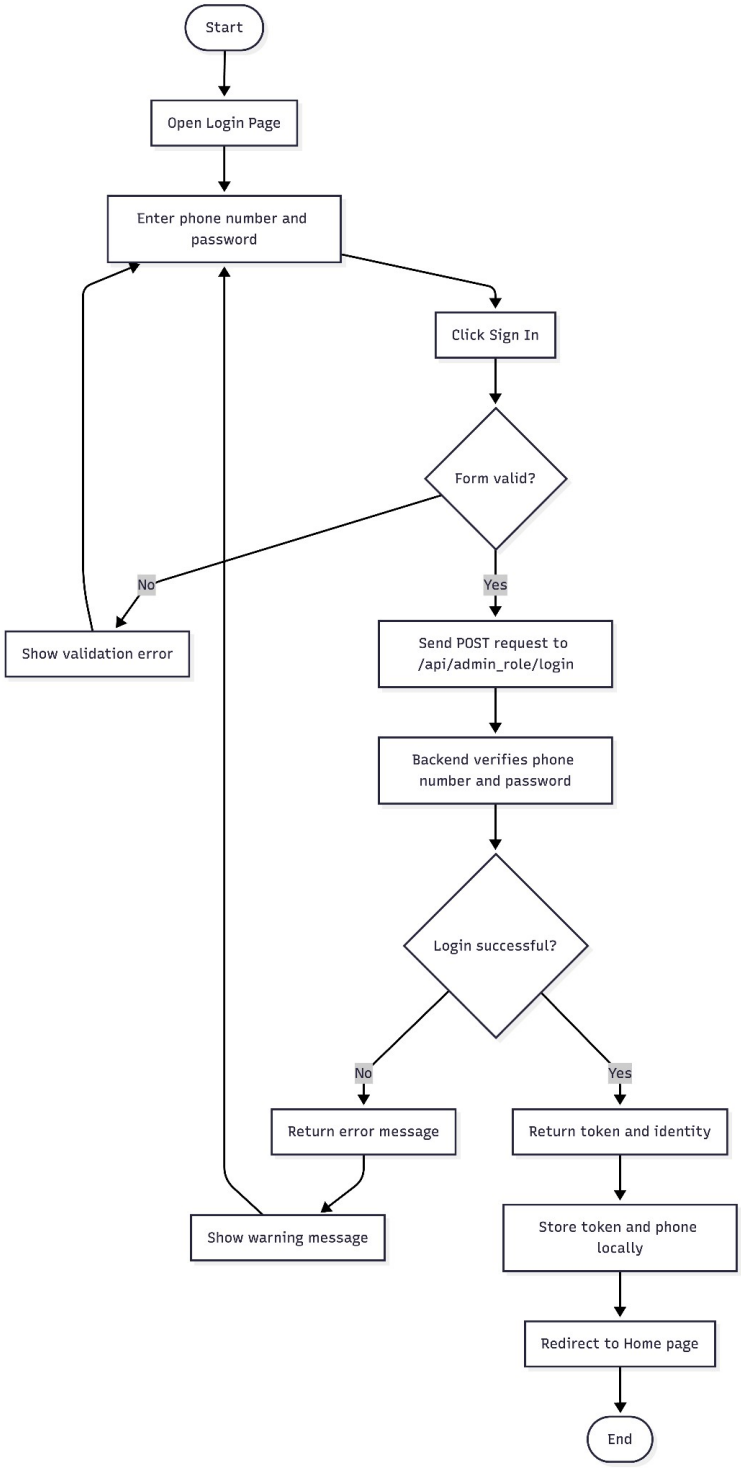

Fig. 2.1.4 Login

2. Notice Management Module (Success Criterion 2)

- Wireframe of GUI: Admin dashboard with "Create" button and "Edit/Delete/Withdraw" actions.
- Function Description: Provides full CRUD operations for school notices and allows manual status changes.
- Algorithm (Flowchart Logic): Input Notice Data -> Validate Permission -> SQL: UPDATE/INSERT Notice Table -> Return Success -> Update UI.

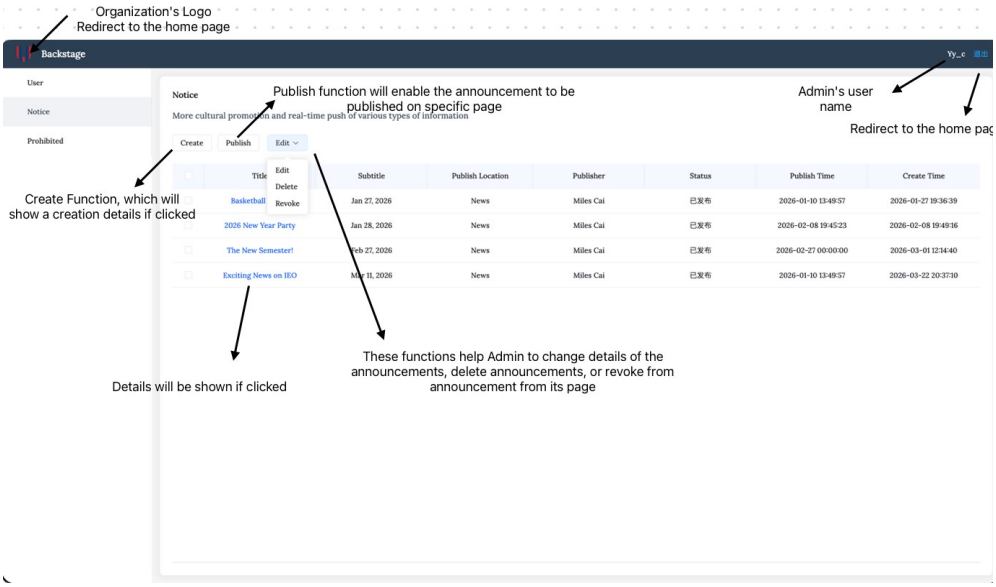

Fig. 2.2.1

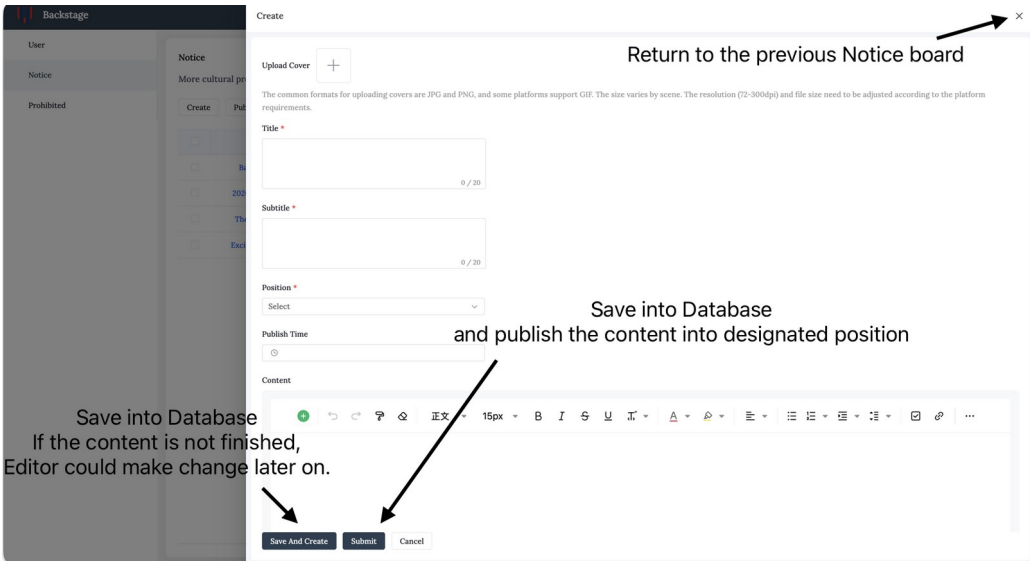

Fig. 2.2.2

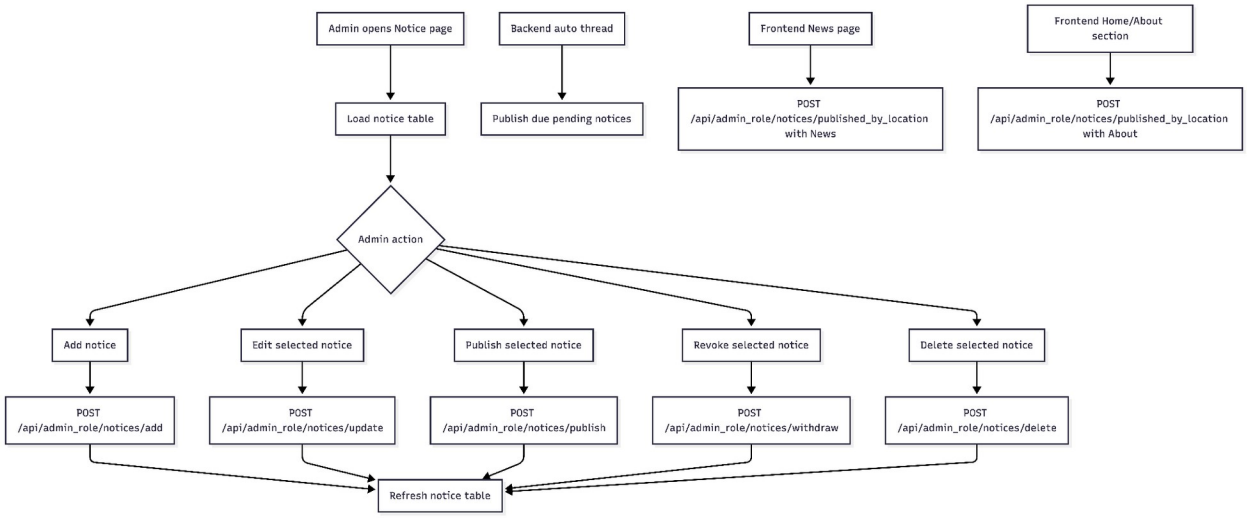

Fig. 2.2.3 Overall Module Structure

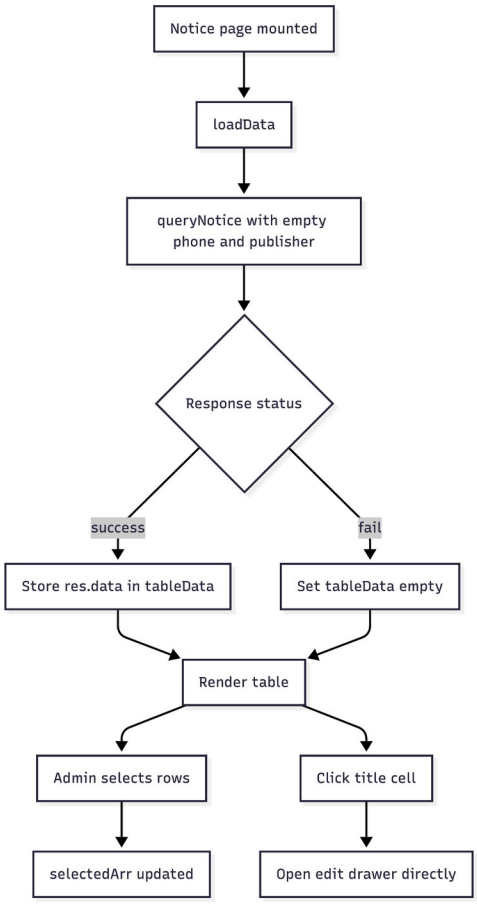

Fig. 2.2.4 Admin List And Select Notices

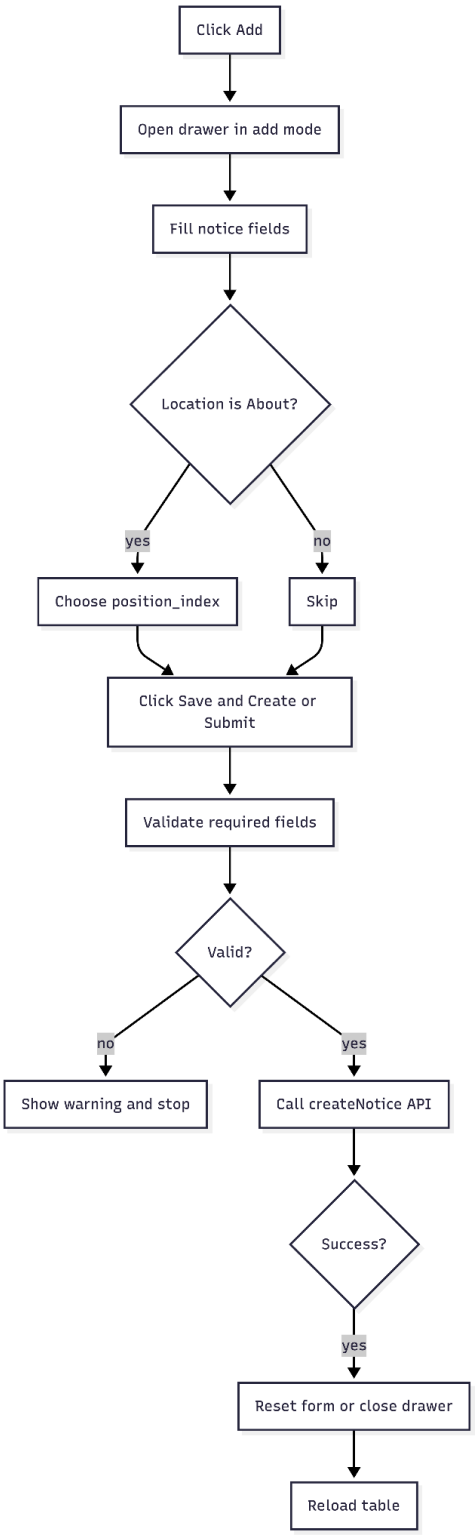

Fig. 2.2.5 Add Notice: Frontend Form

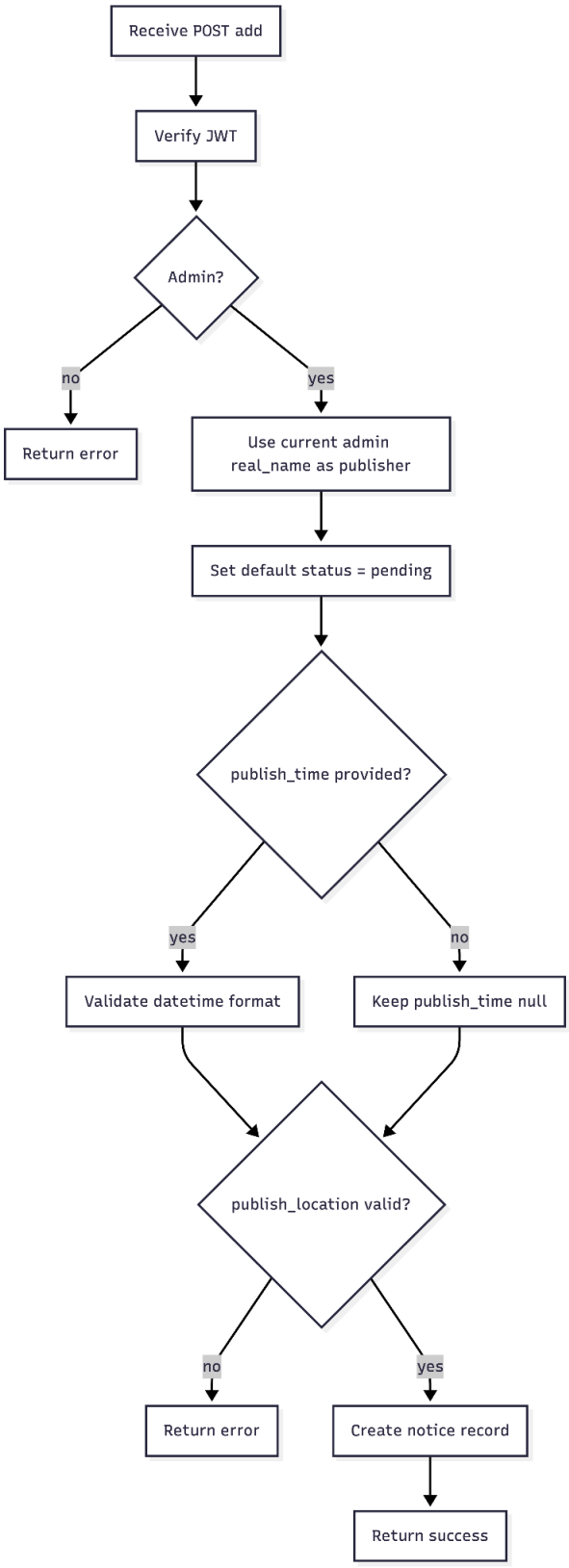

Fig. 2.2.6 Add Notice: Backend Creation

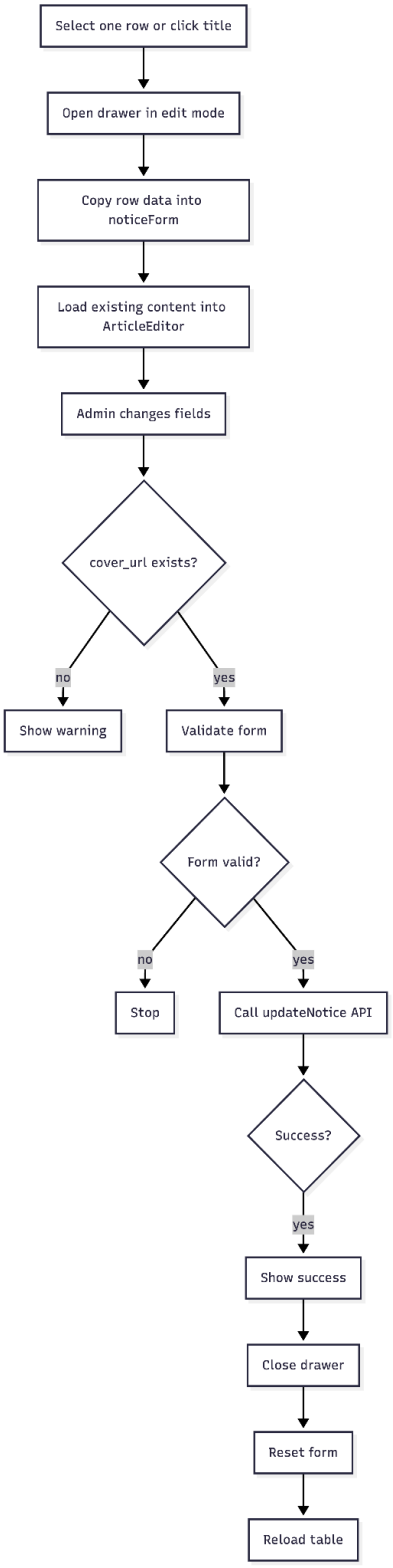

Fig. 2.2.7 Edit Notice: Frontend Submit

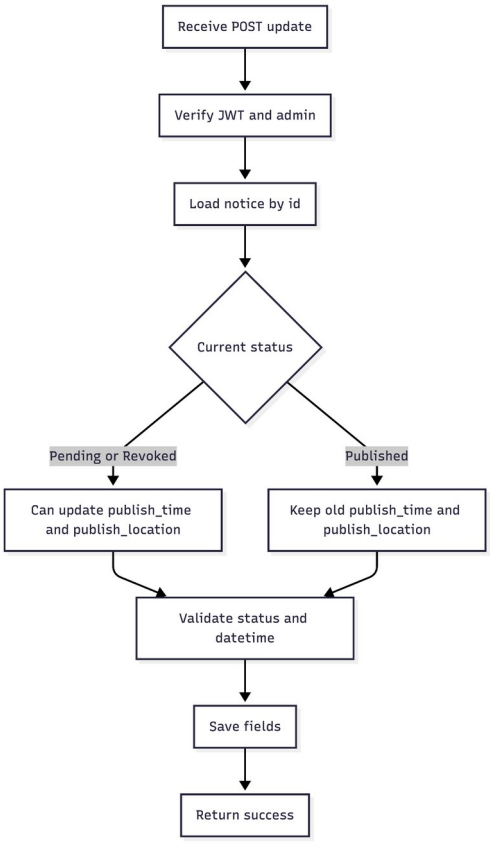

Fig. 2.2.8 Edit Notice: Backend Status Rules

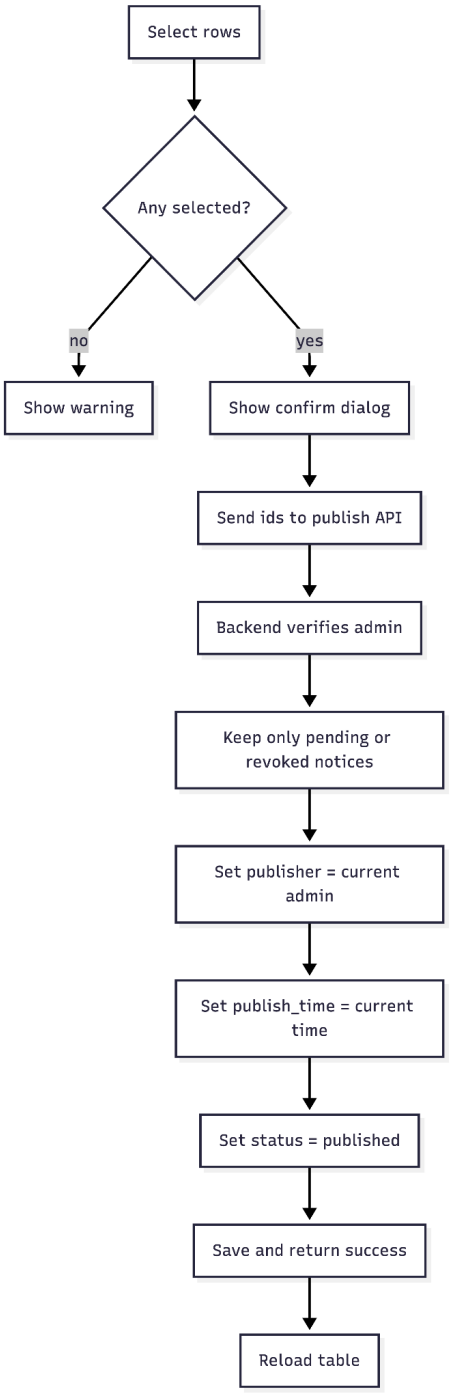

Fig. 2.2.9 Publish Notice

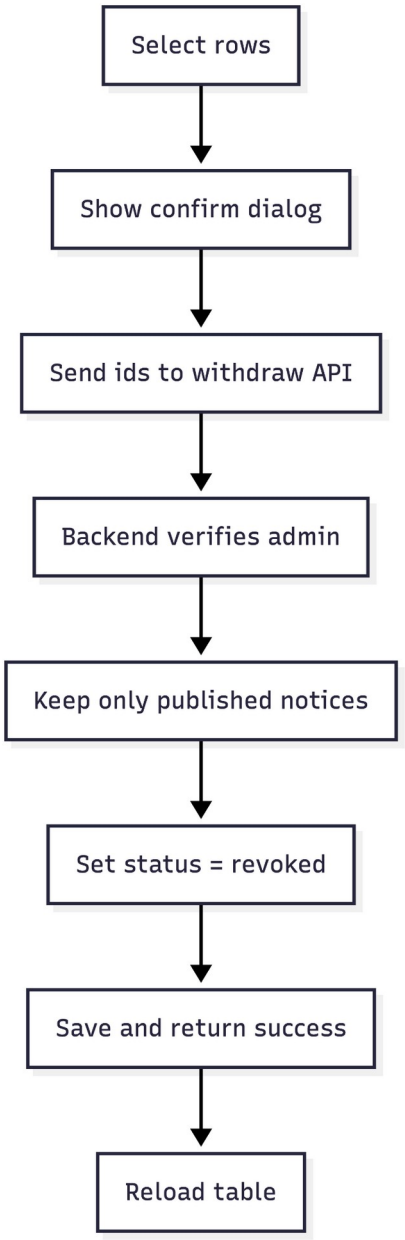

Fig. 2.2.10 Revoke Notice

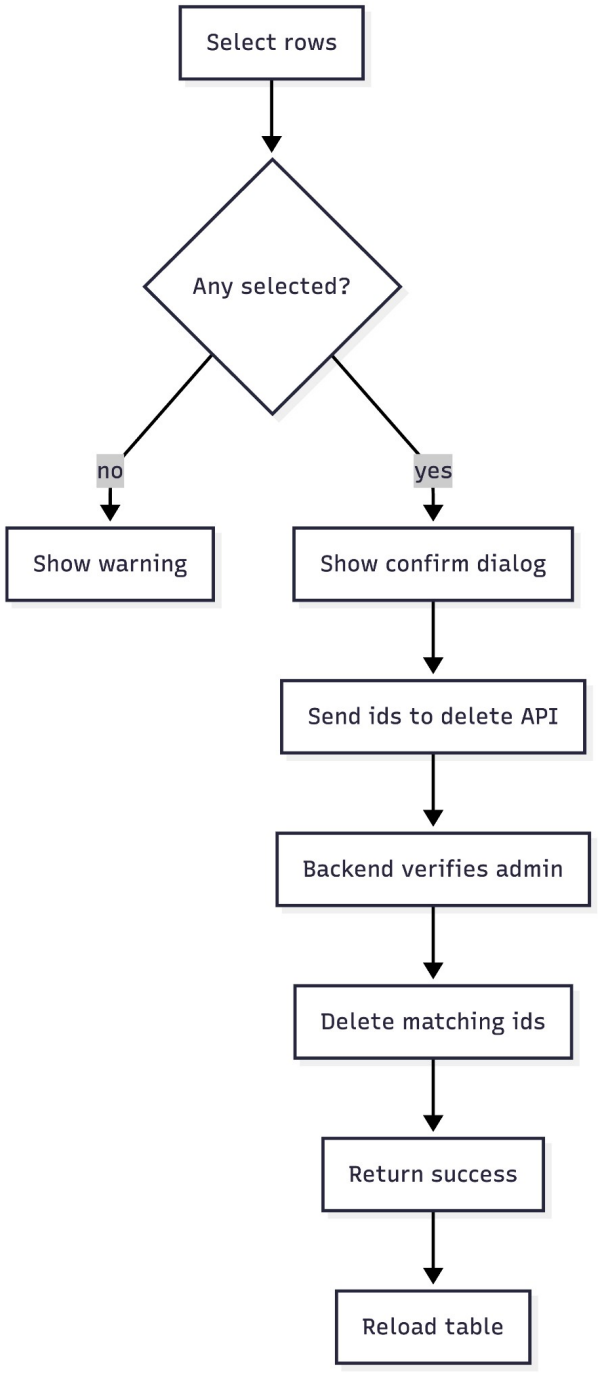

Fig. 2.2.11 Delete Notice

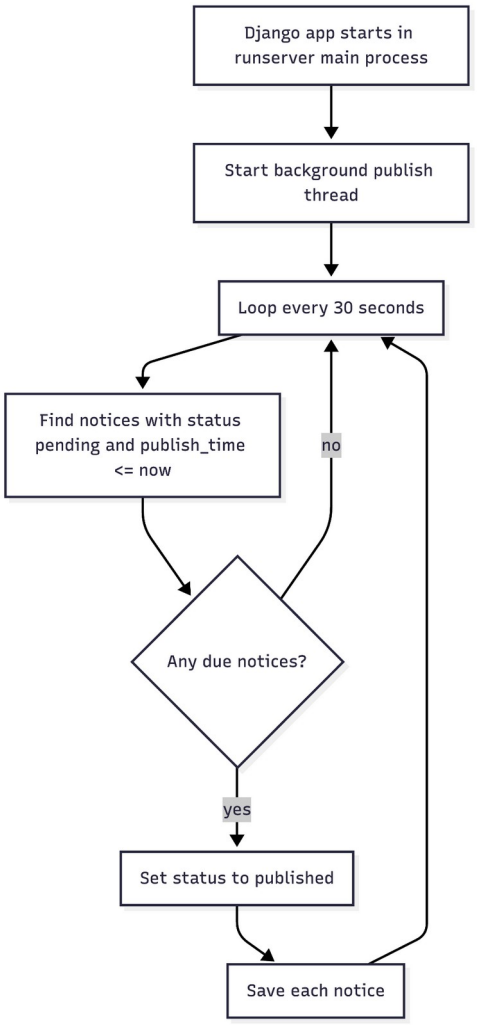

Fig. 2.2.12 Backend Auto-Publish Flow

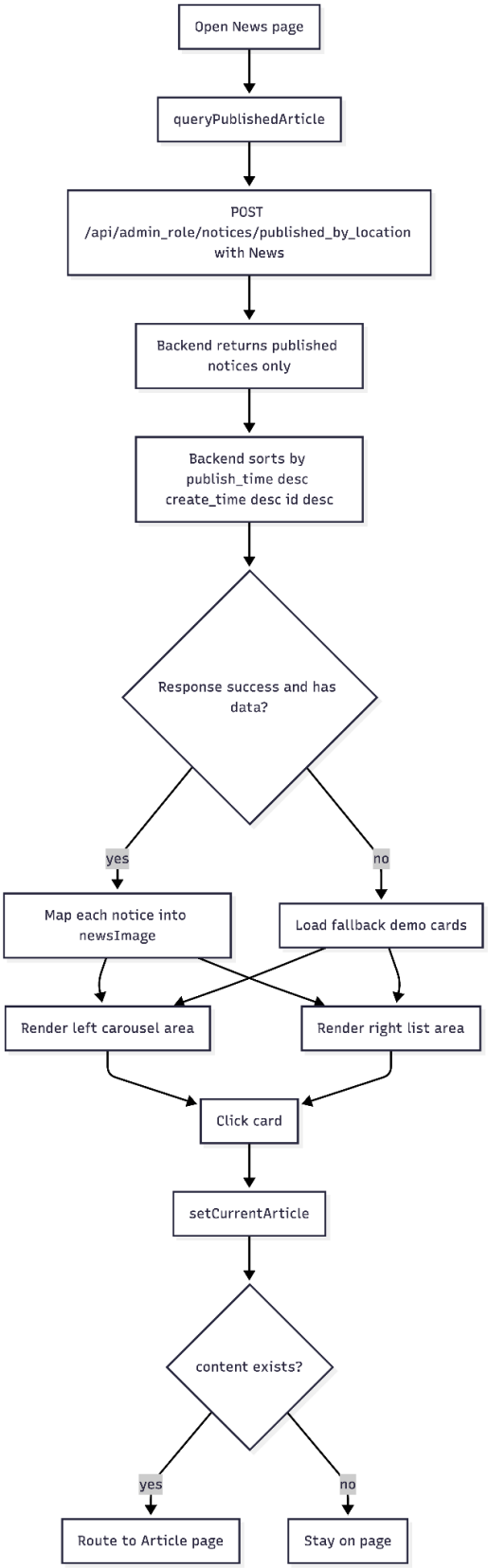

Fig. 2.2.13 Frontend News Page Rendering

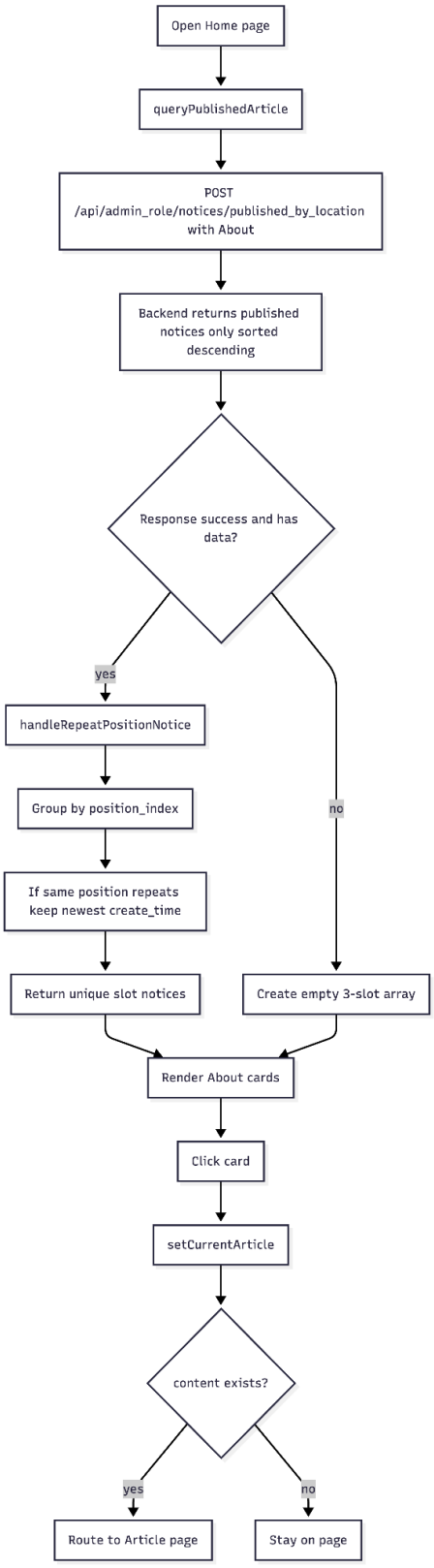

Fig. 2.2.14 Frontend About Page Slot Rendering

3. Automated Publishing Module (Success Criteria 2 and 3)

- Wireframe of GUI: Date-picker on the edit page for "Scheduled Publish Time".
- Function Description: An Asynchronous Thread that auto-updates notice status when the target time is reached.

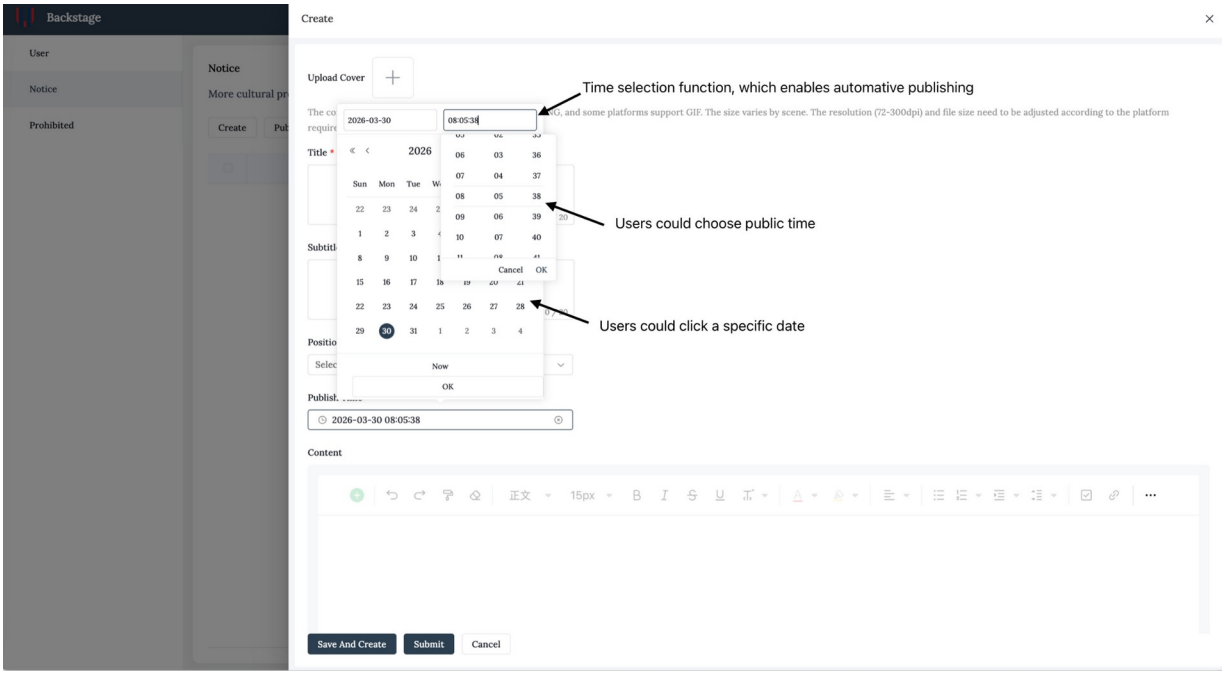

Fig. 2.3.1

Pseudocode for this function:

```
BEGIN Automated_Publishing_Module

    Start module only when the backend server runs in runserver mode

    Create a background thread

    WHILE true DO

        current_time <- get current timezone-aware system time

        notice_list <- query database for notices

            where status = Pending

            and publish_time <= current_time

     FOR each notice in notice_list DO
```

notice.status <- Published

save notice

ENDFOR

sleep for 30 seconds

ENDWHILE

END Automated_Publishing_Module

4. Community Interactive Module (Success Criterion 4)

- Wireframe of GUI: Discussion feed with comment cards and a text submission box.
- Function Description: Handles peer-to-peer communication using DRF Serializers to validate and save student comments. backend validates login token before saving the comment, and author is taken from the authenticated user.
- Algorithm (Flowchart Logic): Student Submits -> Serializer Validation -> Attach Author Name -> SQL: INSERT Comment -> Refresh Feed.

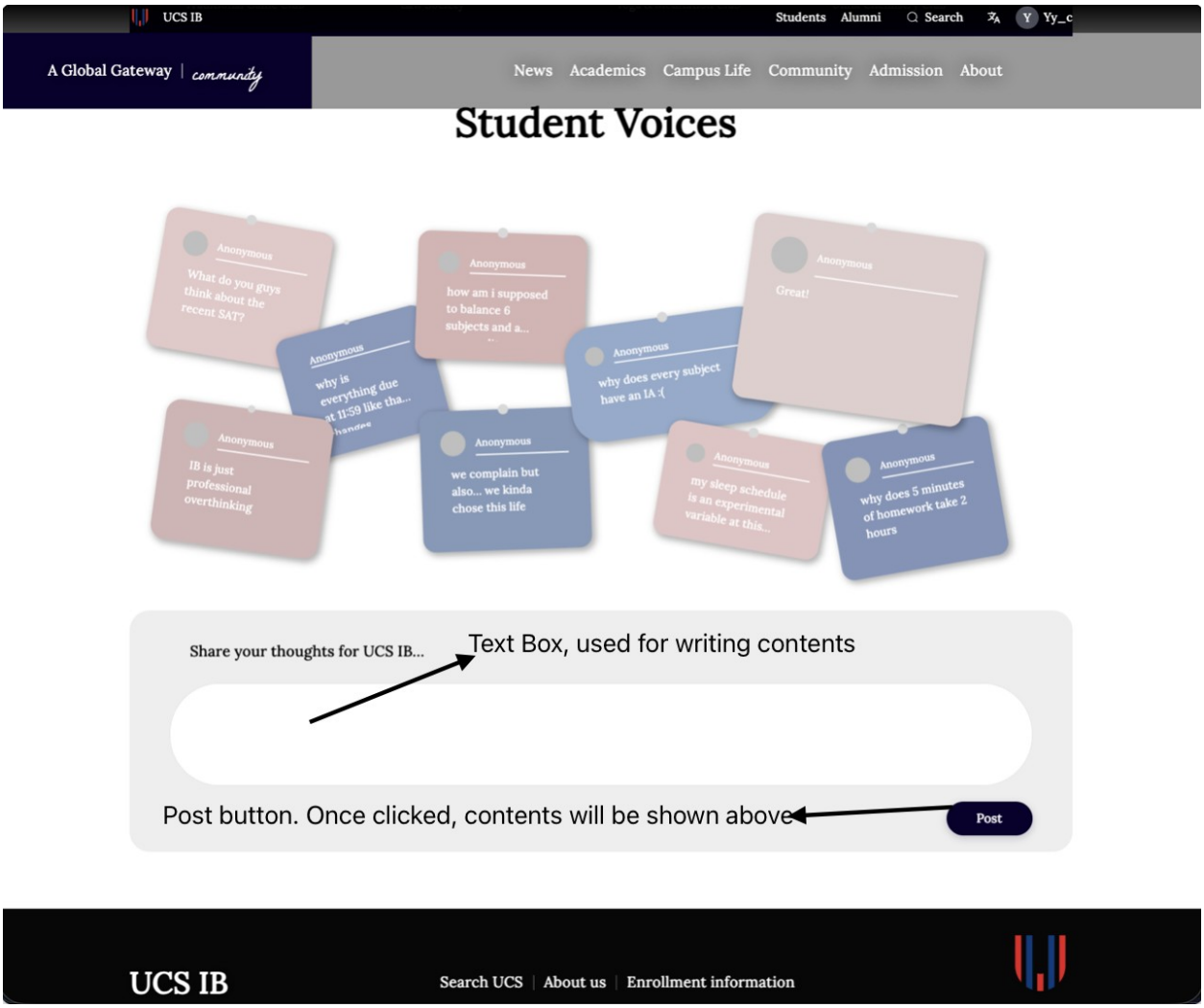

Fig. 2.4.1

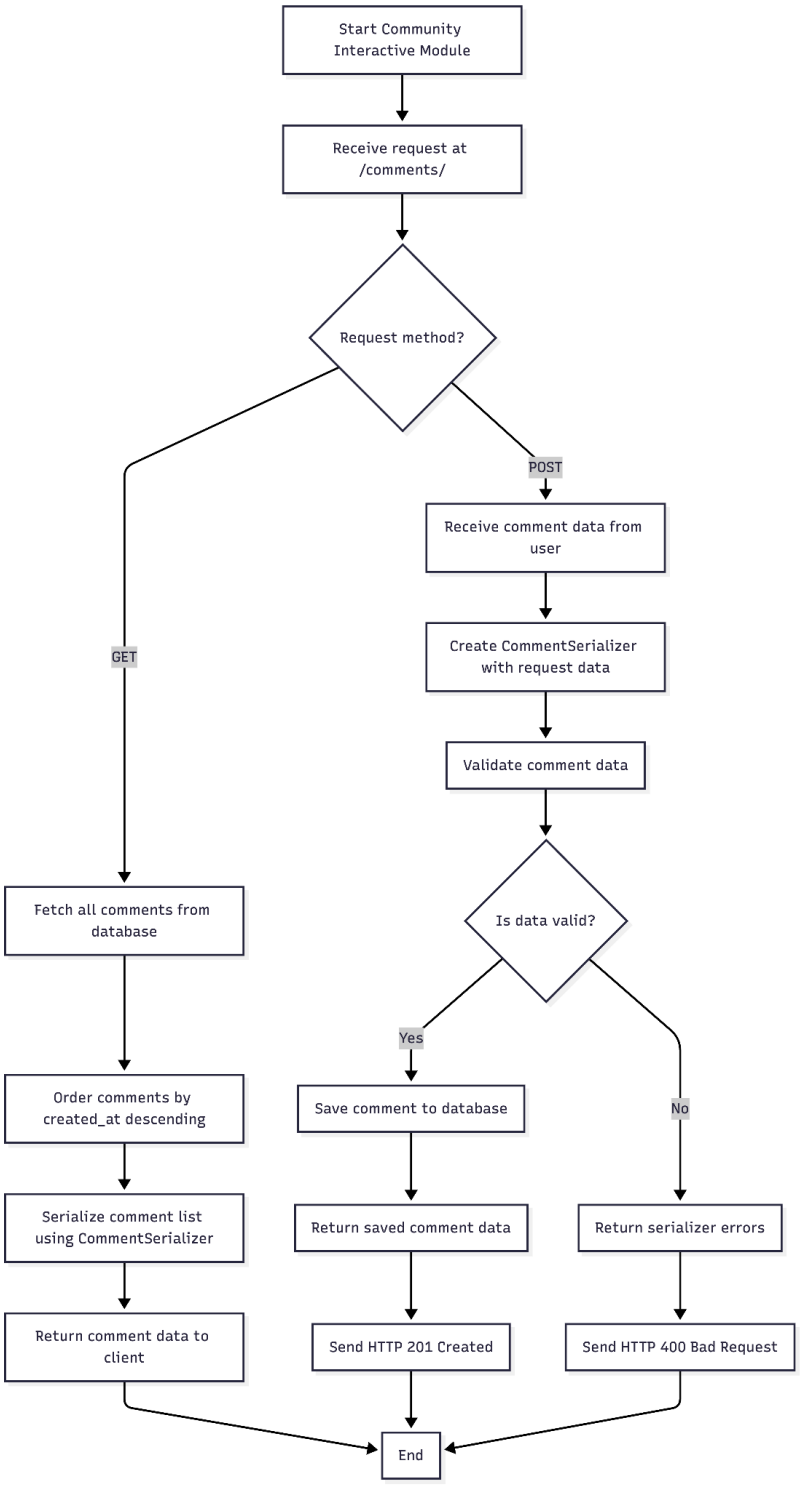

Fig. 2.4.2

5. Multi-Criteria Search Module (Success Criterion 5)

- Wireframe of GUI: Top search bar with a "Category" dropdown (News/Community).
- Function Description: Performs Fuzzy Search using SQL LIKE queries to find keywords in titles and content.

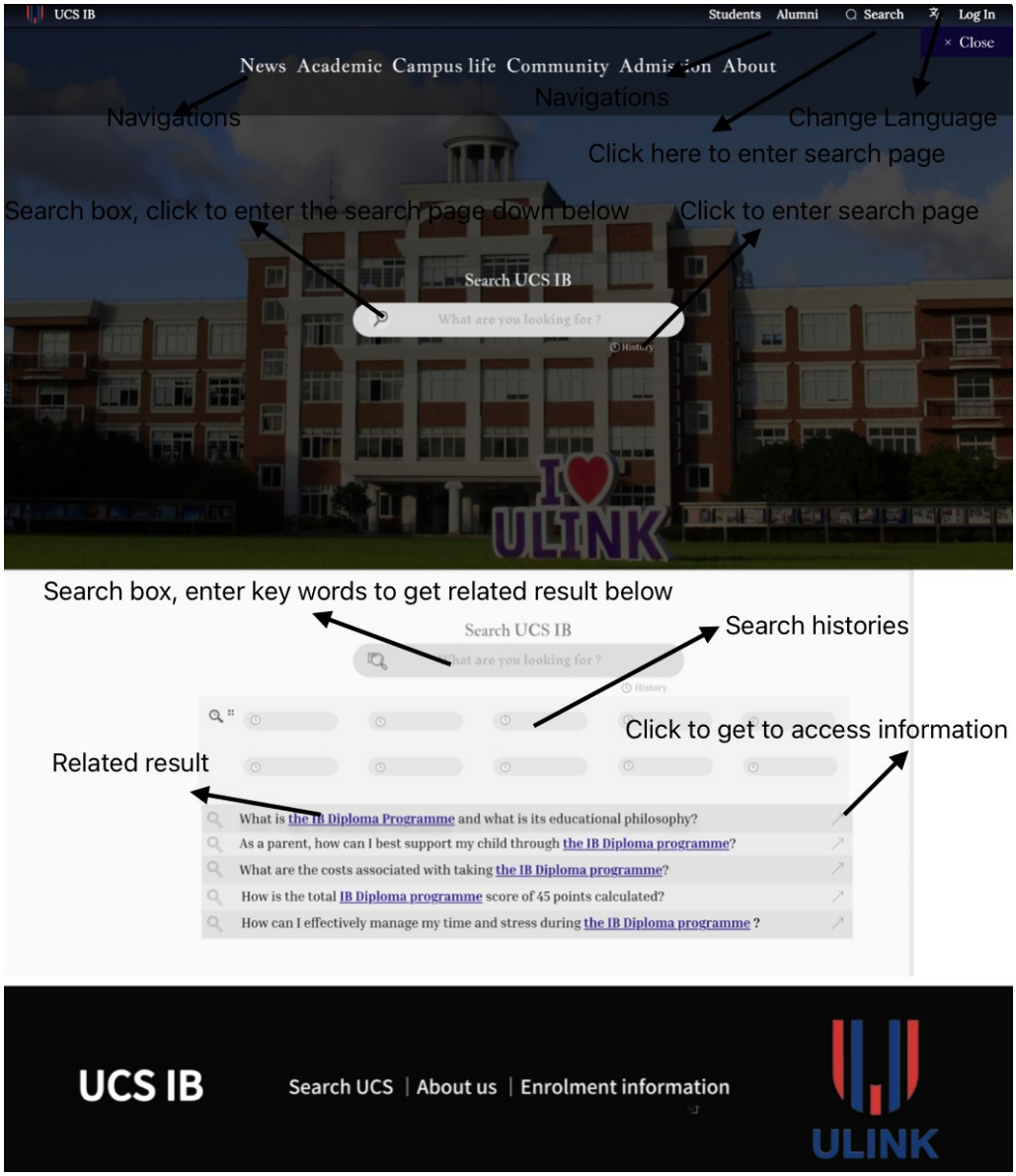

Fig. 2.5.1

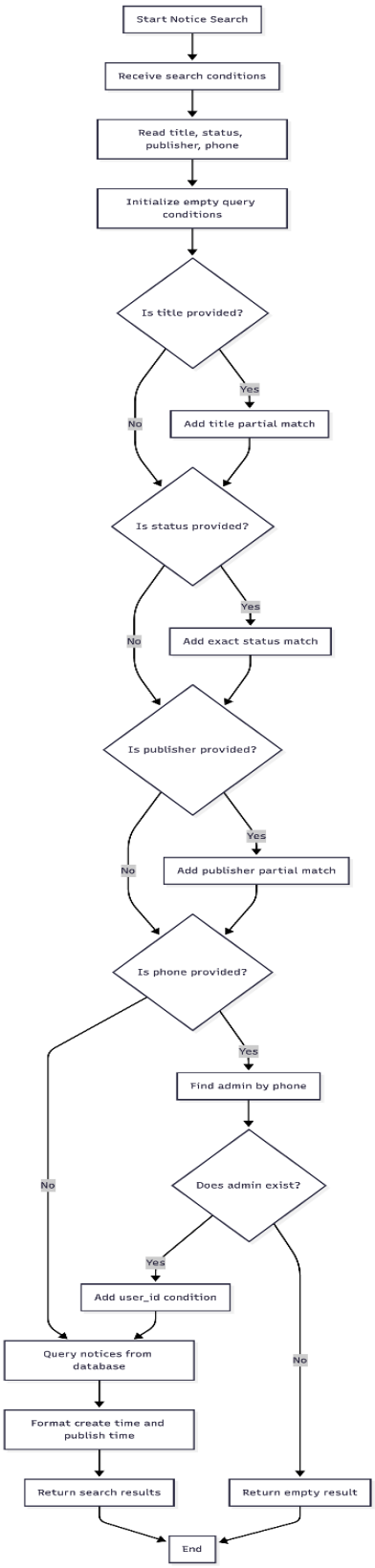
Start Notice Search
Receive search conditions
Read title, status, publisher, phone
Initialize empty query conditions
Is title provided?
Yes
No
Add title partial match
Is status provided?
Yes
No
Add exact status match
Is publisher provided?
Yes
No
Add publisher partial match
Is phone provided?
Yes
No
Find admin by phone
Does admin exist?
Yes
No
Add user_id condition
Query notices from database
Format create time and publish time
Return search results
Return empty result
End

Fig. 2.5.2

6. Mobile Fitting(Success Criterion 6)

3. UML Diagram

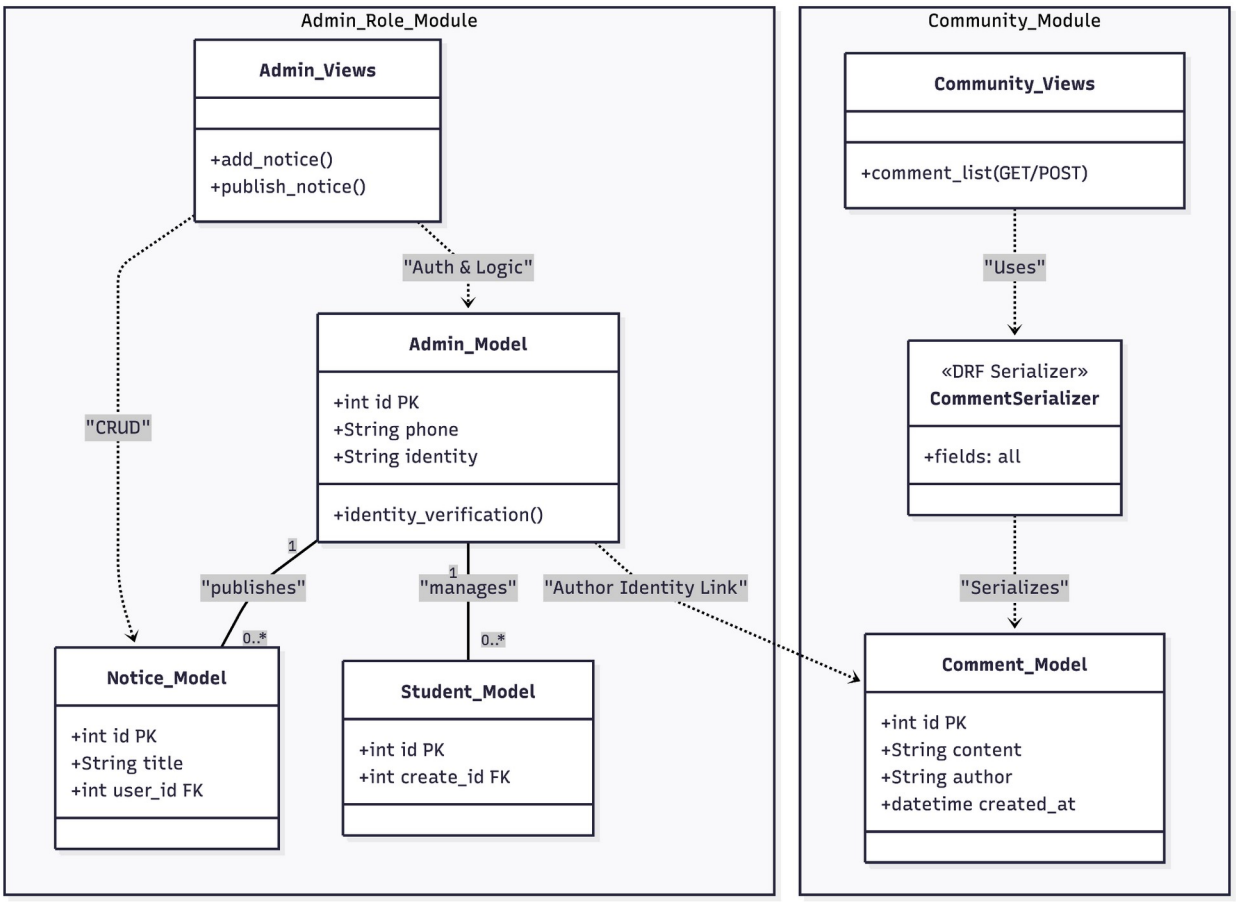

Fig. 3.1

4. Testing Strategy

4.1 Methodology Description

I will perform backend integration testing using Postman to simulate frontend API calls. This allows for direct verification of JSON responses, HTTP status codes, and JWT validation logic. The focus is on ensuring data integrity in the MySQL database and proper error handling for invalid inputs.

4.2 Representative Test Cases

4.2.1 User Authentication & Security (Related SC: #1)

Tested through Postman

Test ID	Related SC	Test Data + Steps	Expected Result
T1	#1	Data: Username: miles_cai, Name: Miles, Phone: 13812345678, PW: SecurePass123, Confirm PW: SecurePass123. Steps: Open Register page → Enter all details → Check "User Agreement" → Click Sign Up.	[Registration Case] New user record is created in the admin_role_admin table; Password is MD5 hashed for security; System redirects to the Sign-In page.
T2	#1	Data: Phone: 13812345678, PW: SecurePass123.	[Login Case] System validates credentials; JWT Token is issued; User is redirected to the Dashboard; User name is displayed in the header.

		Steps: Open Sign-In page → Enter Phone and Password → Click Sign In.	

4.2.2 Notice Management & CRUD (Related SC: #2)

Test ID	Related SC	Test Method	Expected Result
T3	#2	Create a notice with a .jpg cover image.	Image uploaded to Qiniu Cloud; Record saved in admin_role_notice.
T4	#2	Admin edits a notice title and clicks "Update".	DB updates immediately; Dashboard reflects new title.
T5	#4	Set notice publish_time to 1 min in future.	Asynchronous Thread auto-updates status to "Published" at the set time.

4.2.3 Community Comment Board (Related SC: #4)

Test ID	Related SC	Test Method	Expected Result
T6	#4	Student posts: "Is there a club meeting today?".	Comment saved in community_comment with student author link.
T7	#4	Attempt to post a comment	[Invalid Case] DRF returns 400 Bad

		with an empty string.	Requests; record blocked.
T8	#1	Student account attempts to access /admin_add.	[Security Case] Middleware blocks request; returns 403 Forbidden.

4.2.4 Global Search & Data Filtering (Related SC: #5)

Test ID	Related SC	Test Method	Expected Result
T9	#5	Search the keyword "Basketball" in "Announcements".	The system executes icontains query; lists all matching notices.
T10	#5	Search for a non-existent string "abc_123".	[Boundary Case] GUI displays "No results found" message.

4.2.5 T11-T12. Mobile Compatibility (Related SC: #6)

Test ID	Related SC	Test Method	Expected Result
T12	#6	Method: Use Chrome DevTools to simulate iPhone 14 screen width.	[UI Case] Navigation bar collapses into a "Hamburger Menu"; layout switches to a single-column view.
T13	#6	Method: Test "Sign In" button on a mobile touchscreen device.	[UX Case] Input fields and buttons are appropriately sized for touch interaction; text remains readable.